

AI 调试对决实验

最后更新：2026-06-05

1. 实验目的

在 Git Guild 第一轮端到端联调（P4-029）阶段，我们对真实发现并修复的 Bug 分别走「纯人工调试」和「AI 辅助调试」两条路径，分析 AI 在调试环节的价值与局限。

本实验全部取自 [Bug修复日志.md](#) 中的真实条目（BUG-001、BUG-002、BUG-004、BUG-005、BUG-006）。

实验关注的不是单点效率，而是回答三个问题：

- AI 在哪类 Bug 上能快速定位根因？
- AI 在哪类 Bug 上会只修表象、不找根因？
- 人工补充哪些上下文后，AI 的调试建议明显变好？

2. 实验设置

项目	说明
后端	Spring Boot 3.x + Spring Data JPA, H2（测试） / MySQL（演示，本地 Docker Compose）
前端	Vue 3 + Vite
代码托管	本地 Gitea（ http://localhost:3000 ），通过 GiteaAdapter 适配
计时口径	人工：从开始看现象到能口头说清根因；AI：从贴入完整上下文到给出可判断的定位建议

3. 参选 Bug 与选取理由

实验选取三个真实 Bug 作为核心对决案例，分别对应 AI 预期「擅长 / 不擅长 / 中等」三档；另两个纳入对决的 Bug（BUG-004、BUG-005）在第 7 节简要记录。

档位	Bug #	标题	模块	类型	选取理由
AI 擅长	BUG-002	仓库同步未从 Gitea 拉取 Issue	Issue / 仓库同步 / Gitea 适配	行为缺失 / 局部根因	现象明确（同步成功但 Issue 表为空），根因可由单个方法静态推断，验证 AI 在「局部缺失」类问题上的表现
AI 不擅长	BUG-006	Gitea PR 合并后审核台未同步本地 PR 状态	审核台 / Gitea 适配 / PR 同步	跨路径全局时序 / 触发点缺失	同步逻辑只内聚在草稿路径，需跨多条调用链全局推理触发缺口，验证 AI 在「需要全局时序推理」类问题上的表现
中等	BUG-001	重复导入同一 Gitea 仓库生成多条记录、Issue 列表为空	仓库导入与同步	幂等 / 数据一致性	AI 易给出 find-or-save 骨架但易漏边界，验证 AI 方案是否考虑「取最早记录 + 唯一约束」

三个 Bug 均来自 [Bug修复日志.md](#) 的实测记录，发现于 P4-029 第一轮端到端联调。

4. BUG-002：仓库同步未从 Gitea 拉取 Issue（AI 擅长）

4.1 问题现象

- 触发场景：委托人在发布委托面板导入 Gitea 仓库后，点击「读取 Issue 列表」。
- 预期行为：同步后应看到 Gitea 仓库中已创建的 #1 P4-029 demo issue，并可选择该 Issue 发布委托。
- 实际行为：Issue 列表为空，必须手动向 issues 表插入镜像数据后前端才读得到。

关键现象：

```
导入后 repositories:
repository_id=1, source_url=.../gitguild-demo.git, sync_status=SYNCED

同步后 issues 为空:
SELECT * FROM issues WHERE repository_id = 1; -- no rows
```

4.2 对决过程

步骤	纯人工调试	AI 辅助调试
1. 问题描述	成员发现「仓库 SYNCED 但 Issue 空」，去后端找同步逻辑	将现象、复现步骤、syncRepository 与 GiteaAdapter.listIssues 代码一并提供给 AI
2. 定位耗时	约 10 分钟	约 2 分钟
3. 定位准确度	准确：同步接口只 markSynced()，从未调用 listIssues 落库	准确：直接指出同步方法标记了状态却没有拉取并 upsert Issue
4. 修复方案	解析 owner/repo，调 listIssues 后按 (repository_id, external_issue_id) upsert	同向，并额外建议用「查找-或-新建」避免重复同步插入重复行
5. 方案质量	正确	略优：主动提示状态映射（Gitea open → 业务库 OPEN）与去重键
6. 最终修复	采用 AI 的 upsert 写法，人工确认状态映射与契约一致	采纳

4.3 最终修复

- 修改文件：codehost/controller/CodeHostController.java、codehost/domain/CodeIssue.java、codehost/repository/CodeIssueRepository.java、P3ApiDocumentIntegrationTest.java。
- 关键改动：syncRepository 在 markSynced() 前先解析 sourceUrl 得到 owner/repo，调用 giteaAdapter.listIssues(owner, repo)，按 findByRepositoryRepositoryIdAndExternalIssueId 做「查找-或-新建」再 save，并将 Gitea 状态映射为业务库状态。
- 验证：导入 → 同步 → 读取 Issue 链路恢复；mvnw test 通过 (97 passed / 7 skipped)，前端 npm run build 通过。

5. BUG-006：Gitea PR 合并后审核台未同步本地 PR 状态（AI 不擅长）

5.1 问题现象

- 触发场景：用户已在 Gitea 合并 PR #2，回到委托人审核台刷新后再次点击「审核通过」。
- 预期行为：审核台刷新或审核通过前应同步 Gitea PR 状态，把本地 pull_requests.status 从 OPEN 更新为 MERGED，随后允许通过。

- 实际行为：页面仍提示「PR 尚未合并，不能审核通过」；数据库 `pull_requests.status` 仍为 `OPEN`。

关键现象：

```
Gitea: GET /repos/.../pulls/2 -> state=closed, merged=true
本地: pull_requests.pull_request_id=1, status=OPEN
审核: POST /api/v1/submissions/1/reviews decision=APPROVED -> PR_NOT_MERGED

旁证: 手动调 GET /api/v1/quests/1/submission-draft 后, 本地 PR 才变 MERGED
```

5.2 对决过程

步骤	纯人工调试	AI 辅助调试
1. 问题描述	成员发现「Gitea 已 merged，但审核台仍拦截」，怀疑同步没触发	将「审核拦截现象 + ReviewServiceImpl.1 + 审核台 review-queue (首轮未点明同步逻辑
2. 定位耗时	约 18 分钟（需对比三条调用路径才看出缺口）	首轮约 3 分钟给出建议
3. 定位准确度	准确：同步只内聚在 SubmissionDraftServiceImpl.getDraft()，审核队列读取与审核服务都不触发同步	首轮 偏表象 ：建议前端按钮或让用户先打开提交路径未复用同步服务
4. 修复方案	在读队列与审核通过前调用 CodePullRequestSyncService.syncRepositoryPullRequests	补充上下文后改进（见
5. 方案质量	根因级：审核者侧自带同步保障，不依赖冒险家路径	首轮治标：前端按钮只端点」换个入口，未堵
6. 最终修复	采用人工的「读队列 + 审核前双同步」方案	补充上下文后采纳一致

关键转折 —— 人工补充上下文后 AI 明显变好：

第一轮 AI 只看到审核服务与审核台单条路径，默认「同步会在别处发生」，给出前端同步按钮。
人工补充以下三条上下文后再次提问：

- PR 同步逻辑当前**只内聚在** `SubmissionDraftServiceImpl.getDraft()`，即只有冒险家打开提交草稿 / 工作台同步才会触发；

2. 审核台走 GET /api/v1/submissions/review-queue ，只读本地 pull_requests ，不调用同步服务；审核服务 reviewSubmission 只读本地 isMerged() ；
3. 要求审核者侧也有同步保障，不能依赖冒险家先打开草稿。

AI 随即把根因修正为「同步触发点缺失」，给出与人工一致的方案：在 SubmissionServiceImpl.listReviewQueue() 读队列时、以及 ReviewServiceImpl.reviewSubmission() 通过前，对关联仓库调用 CodePullRequestSyncService.syncRepositoryPullRequests() 。

5.3 最终修复

- 修改文件：review/service/SubmissionServiceImpl.java 、 review/service/ReviewServiceImpl.java 。
- 关键改动：读审核队列时先对队列中成果关联仓库去重后同步（syncReviewQueuePullRequests），使审核台刷新能读到 Gitea 最新 PR 状态；提交 APPROVED 前再次同步关联仓库 PR，避免页面缓存状态过期导致误拦截。
- 验证：SubmissionServiceImplTest 覆盖读队列触发同步、ReviewServiceImplTest 覆盖审核前从 OPEN 同步到 MERGED 后放行；mvnw test 通过（101 passed / 7 skipped）。

6. BUG-001：重复导入同一仓库生成多条记录、Issue 列表为空（中等）

6.1 问题现象

- 触发场景：委托人在发布委托面板重复点击「导入仓库并使用」。
- 预期行为：同一 hostType + sourceUrl 重复导入时返回已有仓库，前端复用同一 repositoryId 。
- 实际行为：每次都新增一条 repositories 记录、生成新 repositoryId ；Issue 镜像仍挂在第一次导入的 repositoryId=1 上，前端自动填入新 id 后读取 Issue 为空。

关键现象：

```
repositories:
1 gitguild-demo .../gitguild-demo.git SYNCED
2 gitguild-demo .../gitguild-demo.git SYNCED    <- 重复导入新增
3 gitguild-demo .../gitguild-demo.git SYNCED

issues: 仅 repository_id=1 有镜像；前端填入 repositoryId=2/3 后列表为空
```

6.2 对决过程

步骤	纯人工调试	AI 辅助调试
1. 问题描述	成员看到「仓库编号递增、Issue 空」，定位导入接口	将「重复导入现象 + <code>importRepository</code> 代码 + <code>repositories/issues</code> 表结构」提供给 AI
2. 定位耗时	约 9 分钟	约 2 分钟
3. 定位准确度	准确：导入直接 <code>save(new ...)</code> ，未按 <code>hostType + sourceUrl</code> 查重	准确：指出导入缺少幂等查找
4. 修复方案	先查已有记录、命中则复用；并建议加 <code>(host_type, source_url)</code> 唯一约束	给出标准 <code>find-or-save</code> 骨架
5. 方案质量	根因级，并考虑「取最早记录」让 Issue 镜像可读	首轮遗漏两点边界： 未取最早 <code>repositoryId</code> ，也未提唯一约束做 DB 层根治
6. 最终修复	采用「取最早记录 + 复用」，唯一约束列为后续迁移项	补充上下文后采纳「取最早记录」实现

AI 首轮 `find-or-save` 用任意一条命中记录即可；人工补充「已产生多条记录、Issue 镜像挂在最早的 `repositoryId=1`」后，AI 才补全「按 `repositoryId` 升序取最早一条」并认同唯一约束。属于「方向对但边界不全」的中等档。

6.3 最终修复

- 修改文件：`codehost/controller/CodeHostController.java`、
`codehost/repository/CodeRepositoryRepository.java`、
`P3ApiDocumentIntegrationTest.java`。
- 关键改动：导入时用
`findFirstByHostTypeAndSourceUrlOrderByRepositoryIdAsc(hostType, sourceUrl)` 查
已有记录，命中则复用最早的 `repositoryId`，不再重复插入；唯一约束 `(host_type,`
`source_url)` 列为后续数据库迁移建议。
- 验证：重复导入返回同一 `repositoryId`，Issue 列表恢复；`mvnw test` 通过 (97 passed /
7 skipped)。

7. 实验记录表

Bug #	Bug 描述	纯人工定位耗时	AI 辅助定位耗时	AI 定位是否准确	AI 修复方案是否可用	最终方案来源
BUG-002	仓库同步未拉取 Issue	~10 min	~2 min	准确（首轮即对）	可用，且略优于人工初版（去重键 + 状态映射）	AI 方案 + 人工对齐契约
BUG-006	PR 合并后审核台未同步	~18 min	首轮 ~3 min（偏表象），补上下文后准确	首轮不准（只给前端按钮），补上下文后准确	首轮不可用，补上下文后可用	人工根因 + AI 补充实现
BUG-001	重复导入生成多条记录	~9 min	~2 min	准确	基本可用，遗漏「取最早记录 / 唯一约束」边界	AI 骨架 + 人工补边界

8. 分析

8.1 AI 在哪类 Bug 上表现较好

- 现象明确、根因局部、可静态推断的问题（BUG-002、BUG-005）：同步方法没拉数据、后端已返回错误码却缺前端反馈，AI 顺着单条调用链或单个方法即可快速定位，并常给出比人工初版更周到的细节。

8.2 AI 在哪类 Bug 上表现较差

- 需要跨多条调用路径做全局时序推理的问题（BUG-006、BUG-004）：当输入只覆盖单条路径，有其它读取路径未复用的信息时，AI 默认同步会在别处发生，给出前端同步按钮、手动刷新等表象修复，无法识别触发点缺失的根因。

8.3 AI 是否出现只修表象、不找根因的情况

- 出现，集中在 BUG-006：首轮建议前端加同步按钮，只是把手动绕行换个入口，未触及读队列与审核前未复用同步服务的根因。
- BUG-001 属于方向对但边界不全：找到了缺幂等的根因，但首轮遗漏边界，属于半根因。

8.4 人工提供哪些上下文后，AI 的调试建议明显变好

对 BUG-006，补充以下三类上下文后，AI 从表象修复跃升为根因修复：

1. **代码事实**：明确“PR 同步逻辑当前只内聚在 `SubmissionDraftServiceImpl.getDraft()`”，而非散落各处。
2. **路径事实**：说明审核台走 `review-queue` 只读本地表、审核服务只读本地 `isMerged()`，两者都不触发同步。
3. **约束要求**：要求“审核者侧自带同步保障，不依赖冒险家先打开草稿”，堵死前端按钮类治标方案。

9. 结论

- AI 在调试中是**高效的初筛与定位助手**，尤其擅长现象明确、根因局部、错误码等问题
- AI 的主要短板是**跨路径全局时序推理**，在缺乏“同步逻辑落点 / 哪些路径未复用”上下文时倾向给出表象修复
- 最佳实践是 AI 定位 + 人工根因校验：让 AI 快速缩小范围，由人工确认是否触及根因、是否覆盖并发与跨路径边界，再回喂上下文让 AI 完善实现。

10. 调试阶段 Prompt 最佳实践

1. 一次性提供“现象/报错 + 复现步骤 + 相关代码 + 表结构 / 约束现状”等上下文
2. 显式标注关键事实：同步/校验逻辑当前落在哪条路径、哪些读取路径没复用、是否并发、是否重复投递。
3. 提出根治要求并堵死治标路径（如「不要只加前端按钮」「要在服务端 / DB 层兜底」）。
4. 与 P4-005 接口契约、P3 状态机对照，确认错误码、状态流转一致后再合入。